Agent Plan Executor

Run an LLM's plan as a DAG: in parallel, in order, with failures contained

Tracks: Graphs (topological sort, cycle detection, DP on a DAG) and concurrency (C# async, channels, cancellation) **Reading time:** about 40 minutes, longer if you try it yourself first **Code:** `src/AgentDag/Plan.cs`, `src/AgentDag/DagExecutor.cs`. **Tests:** 23, run with `dotnet test`

---

0. The problem

> "Our agent turns a user request into a plan: a list of steps, each with the steps it depends on. A step is a tool call or a model call, for example `retrieve`, `score`, `web_lookup`, `write_dossier`. Write the component that runs the plan. Steps should run in parallel when they can, but never more than **K** at a time, because our tools have quotas. A step gets its parents' outputs. Decide what happens when a step fails, hangs, or the user cancels."

Try it before you read on. It sticks far better if you've struggled with it first. Hints, if you need them:

▼ Hint 1
A step can start when its *count of unfinished parents* reaches zero. What data structure holds the steps whose count is zero?

▼ Hint 2
If many tasks decrement shared counters, you need locks. Can a **single** loop own all the counters while workers only report "I'm done"?

▼ Hint 3
Running "level by level" with `Task.WhenAll` is correct but slow. Why? (See section 6.1.)

---

1. Why this problem exists: where companies run this exact algorithm

The pattern is a **dependency count plus a ready queue**, which is Kahn's topological sort run live. It is the engine under most systems that run "work with dependencies":

Company / system	What the graph is	How the algorithm shows up	Source
Meta: PyTorch autograd	The backward graph of every training step	The engine first counts dependencies for every node, then queues a node's backward job once all its dependencies are met. That is Kahn's algorithm, run on multiple threads, on every `loss.backward()`	PyTorch distributed autograd design; PyTorch dev on the engine
LangChain: LangGraph (agent frameworks)	An agent's nodes and edges	Nodes ready together run in parallel within a *superstep*, then synchronise (inspired by Pregel). `Send` fans out dynamically; `max_concurrency` limits parallelism. *Same problem, with K*	LangGraph Pregel reference

Company / system	What the graph is	How the algorithm shows up	Source
Microsoft: Azure Durable Functions / Durable Task (C#)	An orchestration's activities	The official **fan-out/fan-in** pattern: start activities without awaiting, then `await Task.WhenAll(...)`. The orchestrator is *replayed* from history, so it must be deterministic	Microsoft Learn: fan-out/fan-in pattern
Netflix: Conductor	Microservice workflows as a JSON blueprint, with fork/join, decisions and sub-workflows	A state-machine service, the *Decider*, combines the blueprint with the current state, works out the next state and schedules tasks. It orchestrated 2.6M+ process flows at the time it was open-sourced	Conductor OSS docs; coverage of the open-source release
Uber: Cadence, then Temporal	Durable workflows	Built at Uber because teams spent most of their time stitching message queues together. Temporal began as a fork of Cadence; its event history makes runs resumable after crashes (see follow-up F3)	Temporal README; SE Daily with Cadence's creator
Airbnb: Apache Airflow	Data and ML pipelines as Python DAGs	The scheduler parses DAGs and hands runnable tasks to workers. *Pools* and per-DAG limits are its K; *trigger rules* are its failure policy (F8)	Apache Airflow; Airflow docs
Google: Blaze / Bazel	The build action graph	Actions run in parallel once their inputs exist, with results cached by input hash. Bazel is "what we use to build the large majority of software within Google"	Google Open Source Blog, 2015
Google: TensorFlow	The dataflow graph of ops	An op runs when its inputs are ready, and independent ops run in parallel across devices	TensorFlow, OSDI '16
OpenAI: Ray (Anyscale)	Task graphs built from futures	A task runs when the futures it depends on resolve. OpenAI said publicly it uses Ray to train its largest models	Anyscale: Ray Summit 2022 LLM stories; HPCwire

Why it matters more now: every agent framework is turning from a loop into a graph. The questions that matter in production are now graph and concurrency questions: whether the plan is valid, how fast it can run, what happens when step 7 of 12 times out, and how you stay under a quota of 5 GitHub calls at a time.

---

2. Questions that shape the design (and the answers assumed here)

Ask	Why it changes the design	Assume
Can the plan contain a cycle?	Decides whether we validate or support loops	No: reject it and **name the cycle** (planner LLMs do produce loops)
Does K apply to the whole run or to each tool?	A global semaphore vs per-resource queues	Global K; per-resource limits are F2
What happens to the rest of the run when a step fails?	Skip descendants vs cancel everything	Configurable, default **skip descendants** (partial results)
Retries? Timeouts?	Changes the per-node loop	Yes, both, configurable; retry only *retryable* errors
Can steps be cancelled?	Need cooperative cancellation	Yes, through a `CancellationToken`
How big are plans?	Recursion depth, heap size	Up to around 100,000 nodes (map fan-outs), so **no recursion**
Do outputs fit in memory?	Ref-count and release outputs	Yes for now; release them in F4

Ask	Why it changes the design	Assume
Must the order be deterministic?	Needed for replay, audit and tests	Yes: break ties by id

---

3. The mental model

How a run goes, step by step:

1. Validate the plan, then count for every step how many parents it is still waiting for (`waitingOn[child]`).
2. Steps waiting for nothing go into a ready queue, ordered by how much work is left behind them.
3. One scheduler loop owns all of this state. While there are free slots and ready steps, it starts the next step as a worker task.
4. Each worker reports back exactly once, through a channel. The loop waits for one report at a time.
5. On success, the loop takes 1 off each child's count, and children that reach 0 join the ready queue.
6. On failure, it skips that step's descendants. With fail-fast, it cancels everything instead.

Three ideas carry the whole problem:

1. **A node is ready when its count of unfinished parents reaches zero.** That is Kahn's algorithm, run live instead of offline.
2. **One owner for all mutable state.** Workers never touch the counters; they send one message each. The scheduler needs no locks, so it can't deadlock and has no lost-update races.
3. **The order you start ready nodes matters once K is small.** Start the node with the longest chain of work behind it first (critical-path-first list scheduling).

---

4. Part A: validate (topological sort with a named cycle)

Algorithm (Kahn), O(V + E):

1. Build a children list and an indegree count for every node.
2. Put every node with indegree 0 in the ready set.
3. Pop a node, append it to the order, and decrement each of its children; a child that reaches 0 goes into the ready set.
4. If `order.Count < V`, the plan has a cycle.

Naming the cycle (the detail that makes it useful):

- "Has a cycle" is useless to the planner LLM that has to repair the plan.
- Every node left with indegree > 0 has at least one parent that is also stuck.
- So walk parent links *within the stuck set* until a node repeats. That loop is a real cycle, found iteratively, so a 100,000-node chain can't overflow the stack. A test proves this.

Why Kahn rather than DFS three-colour? Both are O(V+E). Kahn produces the order the executor needs, doesn't recurse, and matches how the executor runs. DFS is fine if you convert it to an explicit stack.

Validation that LLM-written plans make necessary:

- duplicate ids;
- dependencies on unknown nodes (the LLM made up a tool);
- self-loops;
- duplicate edges (`b` depends on `a` twice must not make `b` wait for two completions);
- negative latency estimates.

---

5. Part B: how fast can it possibly go? (DP on a DAG)

Critical path: `finish[v] = latency[v] + max(finish[parent])`, computed in topological order, O(V+E).

- It is the run time with unlimited workers, so **if the critical path breaks the SLO, more workers won't help; the plan has to change.**
- In the test, a 2 s web lookup sets a 2.9 s critical path even though the scoring chain has more steps.

Bottom level: `bl[v] = latency[v] + max(bl[child])`, computed in reverse topological order. It measures how much work still depends on v. The executor's ready queue is ordered by it.

The bound to quote (Graham / Brent). For any greedy list schedule on K workers:

```
T <= W / K + L
```

Here W is the total work, K is the number of workers and L is the length of the critical path. In words: you're never worse than "perfectly shared work, plus the longest chain". Critical-path-first ordering usually does much better than plain FIFO when K is small.

---

6. Part C: the concurrent executor

6.1 Three designs, and why I chose the third

Design	Sketch	Problem
Level by level	Group nodes by depth; `await Task.WhenAll(level)` for each level	**Barrier waste.** One slow node in level 1 holds up every node in level 2, even ones that don't depend on it. With a 2 s lookup beside 50 ms scoring calls, the whole next level waits 2 s. *The most common first attempt.*
Task per node, awaiting its parents	`tasks[v] = Run(async () => { await Task.WhenAll(parents); await sem.WaitAsync(); try { ... } finally { sem.Release(); } })`	Elegant and correct for the happy path. But you can't choose *which* ready node gets the next slot (the semaphore wakes waiters FIFO), failures arrive as exceptions in many places, and every node's task exists from the start
Scheduler loop + completion channel (my choice)	One loop owns the counters and ready heap, starts up to K, and awaits one completion at a time	A little more code, but priorities, failure policy, cancellation and accounting all live in **one place**

6.2 The loop (from `DagExecutor.cs`)

```
while (true)
{
    stopping |= runCts.IsCancellationRequested;
    while (!stopping && running < _o.MaxConcurrency && ready.TryDequeue(out var id, out _))
    {
        var inputs = plan.Parents[id].ToDictionary(p => p, p => outputs[p]);
        running++;
        _ = RunNodeAsync(id, work[id], inputs, runCts.Token, completions.Writer); // always reports
exactly once
    }
    if (running == 0) break;

    var done = (await completions.Reader.ReadAsync()).Result;   // no token: never abandon in-flight work
    running--;
    results[done.Id] = done;
    switch (done.Status)
    {
        case NodeStatus.Succeeded:
            outputs[done.Id] = done.Output;
            foreach (var c in plan.Children[done.Id])
                if (--waitingOn[c] == 0 && !results.ContainsKey(c)) MakeReady(c);
            break;
        case NodeStatus.Failed when _o.FailurePolicy == FailurePolicy.FailFast:
            stopping = true; await runCts.CancelAsync(); break;
        case NodeStatus.Failed:
            foreach (var d in plan.Descendants(done.Id)) results.TryAdd(d, Skipped(d)); break;
    }
}
```

6.3 Invariants (the properties that make it correct)

1. `running` **stays between 0 and K**, always. The loop is the only place that changes `running`.
2. **A node starts only after all its parents have *succeeded*.** It is enqueued only when `waitingOn` reaches 0, and that only happens on success.
3. **Every node starts at most once.** It is enqueued exactly once, because its count reaches 0 exactly once.
4. **Every started node reports exactly once.** `RunNodeAsync` catches everything and writes once, so the loop can wait without a timeout and never hangs.
5. **At the end, every node has exactly one result:** Succeeded, Failed, Skipped or Cancelled. Nothing goes missing.
6. **No shared mutable state between workers.** Each worker gets a fresh copy of its inputs.

6.4 Failure semantics

Event	SkipDescendants (default)	FailFast
A node throws a retryable error (`HttpRequestException`, `IOException`, timeout)	Retry with exponential backoff and full jitter, up to `MaxAttempts`	Same
A node throws a bug (`ArgumentException` and similar)	**No retry**; the node fails; all its descendants are skipped; other branches continue	The node fails; the run token is cancelled; in-flight nodes end as Cancelled; nothing new starts

Event	SkipDescendants (default)	FailFast
A node hangs	The per-attempt timeout (`CancelAfter`) raises `TimeoutException`, which is retryable	Same
The tool ignores its token	`WaitAsync(token)` stops *waiting* on it, so the run continues; the stray work is reported in metrics	Same
The user cancels	In-flight nodes see the token and end as Cancelled; unstarted nodes are Cancelled	Same

6.5 C# concurrency details that matter here

- `Channel<T>` **with** `SingleReader = true` for completions. It is lock-free in practice and keeps the single-owner model honest.
- `Task.Yield()` **at the top of each worker.** A work function that blocks *before* its first `await` would otherwise run on the scheduler's thread and stall the loop.
- **Fire-and-forget only when it's safe.** `_ = RunNodeAsync(...)` is fine *only* because that method can't throw and always reports. Never write `async void`.
- `WaitAsync(token)` **does not stop the work.** It only stops waiting. Real cancellation is cooperative: tools must pass `ctx.Cancellation` into `HttpClient` and similar calls.
- **Dispose every** `CancellationTokenSource` (`using`). A linked CTS registers with its parent and leaks if not disposed.
- `ConfigureAwait(false)` **in library code**, so it can't deadlock against a caller's synchronisation context.
- **Never** `.Result` **or** `.Wait()` on async work. Sync-over-async deadlocks and exhausts the thread pool.
- **For a flat fan-out with no edges**, `Parallel.ForEachAsync(items, new ParallelOptions { MaxDegreeOfParallelism = K }, ...)` is the built-in, but it has no notion of dependencies, which is why it isn't enough here.
- **Retry only idempotent work.** `ctx.Attempt` exists so a tool can reuse the same idempotency key on every attempt, so a retried "charge" or "send email" doesn't happen twice.

6.6 Complexity

	Cost
Validate + order	O(V + E), plus O(V log V) for the deterministic heap
Scheduling	O((V + E) + V log V) heap work. Scheduling overhead is microseconds per node; tool latency (ms to s) dominates
Memory	O(V + E) for the graph and counters, plus every output held until the end. **F4 fixes that**
Skipping descendants	O(V + E) in total in the worst case. Cache descendant sets if many failures are expected

---

7. The tests, named after what they protect

Test	Invariant / behaviour
`Plan_with_a_loop_is_rejected_and_the_loop_is_named`	Cycle detection + the cycle reported as real edges from parent to child
`Hundred_thousand_node_chain_with_a_back_edge_does_not_overflow_the_stack`	Iterative, scales to 100,000 nodes
`Critical_path_is_the_slowest_chain_not_the_longest_one`	DP on a DAG with weights, not step counts
`Never_more_than_K_tools_in_flight_and_K_is_actually_reached`	Invariant 1 *and* real parallelism
`Random_plan_of_300_nodes_runs_each_node_once_and_never_before_its_parents`	Invariants 2 + 3 on a seeded random DAG
`One_candidates_failed_lookup_skips_only_its_own_dossier`	Failures stay contained, partial results come back
`Fail_fast_cancels_the_slow_sibling_and_starts_nothing_new`	Fail-fast semantics
`Rate_limited_tool_succeeds_on_the_third_attempt` / `A_bug_is_not_retried`	Retry classification
`Hung_tool_times_out_and_is_retried_then_fails` / `Tool_that_ignores_its_token_still_cannot_hang_the_run`	Timeouts, including non-cooperative tools
`Manager_cancels_mid_run_and_every_node_is_accounted_for`	Invariant 5 under cancellation
`With_one_slot_the_node_with_the_longest_tail_starts_first`	Critical-path-first priority
`Parallel_run_takes_about_the_critical_path_not_the_sum`	Real speedup, close to the critical path

8. Going further: what real systems need next

F1. "The plan says 'for each shortlisted candidate, run lookup, then dossier'. You don't know how many until runtime."

- *Dynamic fan-out.* Add a `Map` node whose output is a list. When it succeeds, the scheduler **expands** a templated sub-graph per item and rewires the join.
- Validate the new piece before splicing it in, using the same checks as Part A. The sub-graph is acyclic and only points forward, so the whole plan stays acyclic.
- LangGraph's `Send` and Conductor's dynamic fork are the same idea.

F2. "GitHub allows 5 concurrent calls, the LLM 20, the database 50."

- Use **per-resource limits.** Each node declares a resource, and each resource gets its own ready queue and slot count.
- With one shared ready queue, a burst of GitHub nodes would hold every slot while LLM nodes sit idle (*head-of-line blocking*).

- A node needing two resources should take them in a **fixed global order**, so two nodes can never each hold one and wait for the other.

F3. "The process dies 9 minutes into a 10-minute run."

- Make it **durable.** Append every state change (started, succeeded with output reference, failed) to an event log before acting on it. On restart, replay the log to rebuild the counters, and re-run only nodes that never finished.
- Side effects need **idempotency keys** (`runId:nodeId:attempt`).
- This is exactly Temporal's and Durable Functions' model, and it is why their orchestrator code must be deterministic: replay re-runs it.

F4. "Plans with 50,000 nodes run out of memory."

- **Reference-count outputs:** `remainingReaders[v] = children(v).Count`. Decrement it when a child starts (its inputs are copied), and drop the output at zero.
- Store big outputs outside memory (blob store) and pass references, which is how Conductor and Durable Functions handle large payloads.

F5. "Run 10,000 plans across 50 machines."

- Partition **runs**, not nodes. Route by `runId` with consistent hashing so one scheduler owns each run, keeping the single-owner model.
- Node *workers* can be a shared pool behind a queue.
- Ownership moves through a **lease with a fencing token**, so a scheduler that pauses for 40 s can't commit after its replacement has taken over.

F6. "Cap each run at $5."

- Make budgets a resource. **Reserve** the estimated cost before starting a node and settle the actual cost after.
- Refuse to start a node whose reservation doesn't fit. The run then ends with partial results and the reason, rather than overspending.

F7. "Some steps wait for a human to approve, sometimes for days."

- A human gate must **not hold a concurrency slot**. Model it as a `Waiting` state outside `running`.
- Persist the run (F3) and wake it on an external signal. This is Durable Functions' external events, or Temporal signals.

F8. "'Notify the recruiter' should run even if a lookup failed."

- Use **trigger rules per node:** `all_success` (the default), `all_done`, `one_failed`, `none_failed`. These are Airflow's trigger rules.
- In the loop, decrement `waitingOn` on *every* terminal status, not just success, then evaluate the child's rule to decide Ready or Skipped.

F9. "Our critic step needs to loop: write, critique, rewrite."

- Keep the plan a DAG. Put the bounded loop **inside** a node, or unroll it a fixed number of times (`write1, critic1, write2, critic2`).

- Frameworks that allow cycles add a hard limit for exactly this reason: LangGraph's recursion limit, for example.

F10. "One huge run starves everyone else."

- **Fairness between runs:** a weighted fair queue across runs or tenants in front of the shared worker pool.
- Within a run, keep critical-path-first ordering.

---

9. From working code to production

Concern	What I'd add
Observability	One trace per run and one span per attempt (`node.id` , `attempt` , `status` , `error.class` , `queue_wait_ms`). Metrics: ready-queue depth, slot use, retries by class, stray (non-cooperative) work
Back-pressure	Bound the number of runs accepted per scheduler; reject new runs early with a retry-after when overloaded
Security	A node receives *only its parents' outputs* (already true). Tool credentials go through a gateway scoped to the plan's permissions, never in the node's inputs
Testing	Deterministic simulation: inject a fake clock and fake work, then fuzz random DAGs, failure points and cancellation times, checking invariants 1 to 6 after every run
Deployment	Version the node catalogue and the plan schema; pin both per run so an in-flight run never sees a half-deployed change

---

10. Common mistakes, and why they hurt

Topic	Common mistake	What goes wrong	Better
Ordering	Run nodes in the order they're listed	A node runs before its inputs exist	Kahn's algorithm, with a named cycle, a deterministic tie-break and no recursion
Parallelism	`Task.WhenAll` on everything	No limit, so tool quotas are blown and the run gets 429s	Cap concurrency at K
Parallelism	Level by level	Barrier waste: fast branches wait for slow, unrelated ones	A scheduler loop that starts each node the moment its own inputs are ready
Shared state	Many tasks decrementing shared counters	Lost updates: a node never becomes ready and the run hangs silently	One owner of all state; workers report through a channel
Failure	One exception ends everything	Partial results that were fine are thrown away	Retry only transient errors, use idempotency keys, choose a failure policy per run, give every node a final status
Cancellation	Ignored, or only half handled	Orphaned work keeps running and spending quota	Tell timeout apart from cancellation, guard against tools that ignore the token, never abandon in-flight work
Performance	Only Big-O	No idea whether the deadline is even reachable	Critical path as the lower bound, `W/K + L` as the upper bound, critical-path-first ordering

---

11. Run it

```
cd dsa/graphs/agent-plan-executor
dotnet test          # 23 tests: Plan (10) + Executor (13)
```

Requires .NET 10 SDK.

Sources

- PyTorch: Distributed autograd design (dependency counting); Forum: engine queues jobs when dependencies are satisfied
- LangGraph: Pregel runtime reference; Forum: parallel fan-out best practices
- Microsoft: Fan-out/fan-in pattern in Durable Functions (Microsoft Learn)
- Netflix: Conductor OSS docs; Open-source release coverage; Original slides
- Uber / Temporal: Temporal repository; SE Daily: Cadence; Cadence vs Temporal FAQ
- Airbnb / Airflow: Wikipedia: Apache Airflow; Airflow docs
- Google: Bazel reaches beta (Google Open Source Blog); TensorFlow, OSDI '16
- OpenAI / Ray: Anyscale: Ray Summit 2022; HPCwire

Company details come from public posts and docs, as of when they were published. Internal systems change; read them as "publicly described as...".